

Proyecto Final Integrador

DevOps & Cloud Computing

Diseñar, construir y operar un servicio nuevo en AWS, end-to-end. Esta guía describe el alcance, los entregables, los criterios de evaluación y las decisiones técnicas que cada estudiante debe fundamentar para llevar a producción la solución elegida.

1

Objetivo

Llevar un servicio containerizado a producción en AWS con todas las prácticas de un puesto DevOps
Mid: infraestructura como código, pipeline de CI/CD reproducible, observabilidad efectiva y gestión de costos.

El énfasis del proyecto está en **operar** un servicio en producción, no en desarrollarlo. Por eso la aplicación se toma de un proyecto open source con licencia permisiva. El trabajo del proyecto consiste en aprovisionar la infraestructura, construir el pipeline, definir la observabilidad y documentar las decisiones que llevaron a esa solución.

2

Escenarios y aplicaciones de referencia

Cada escenario tiene una aplicación open source recomendada que cubre los componentes mínimos. Las cuatro aplicaciones son self-hostable, tienen imagen Docker oficial, exponen una API HTTP y soportan, de forma nativa o con configuración mínima, métricas en formato Prometheus.

Cada escenario se presenta con: la aplicación recomendada, los componentes mínimos esperados, las métricas relevantes para definir SLOs, la documentación oficial pertinente, una topología sugerida en forma de diagrama y un conjunto de decisiones técnicas que cada estudiante debe tomar y fundamentar en el reporte. Los diagramas ilustran los componentes que la solución espera contemplar; los detalles operativos (cantidad de réplicas, esquemas de IAM, security groups, tags, configuración del ALB, dimensionamiento de la base de datos) se diseñan y justifican como parte del proyecto.

Aviso Una sola elección por proyecto

Elijan un escenario y manténganlo durante todo el desarrollo. Saltar de escenario a mitad de proyecto invalida la mayoría del trabajo previo de infraestructura.

Escenario A — Acortador de URLs

Aplicación recomendada: [Shlink](#) — acortador self-hosted maduro, con REST API documentada, soporte de PostgreSQL y MariaDB/MySQL, un endpoint `/rest/health` para health checks y exportador de métricas Prometheus integrado.

Componentes mínimos a desplegar

- Container de Shlink corriendo `shlinkio/shlink`.
- RDS PostgreSQL como almacenamiento persistente.
- (Opcional) Redis vía ElastiCache si se habilita el caché distribuido para deployments con múltiples tasks.

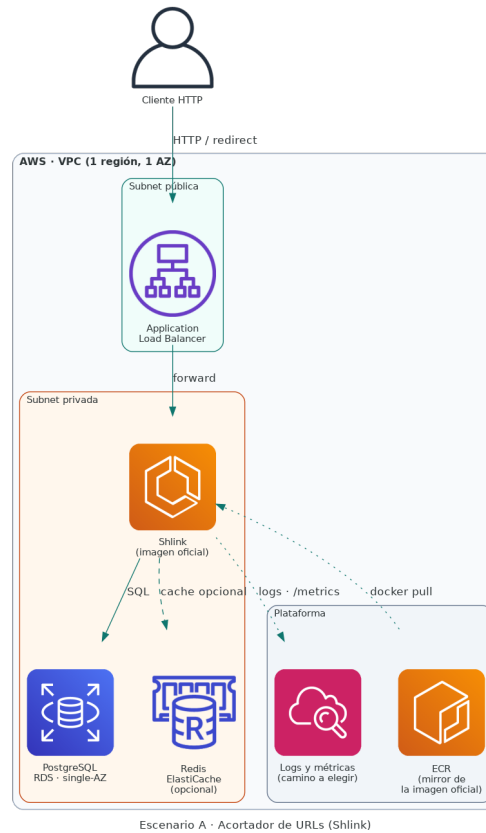
Métricas relevantes para SLOs

Latencia del redirect, ratio de aciertos del cache, tasa de errores 4xx (códigos inválidos), tasa de errores 5xx.

Documentación clave

- Variables de entorno y configuración: <https://shlink.io/documentation/install-docker-image/>
- Endpoints de la REST API: <https://shlink.io/documentation/api-docs/>
- Health check y métricas: <https://shlink.io/documentation/advanced/health-check/>

Topología sugerida



Topología sugerida del escenario A. Los detalles de réplicas, IAM, security groups y tagging son decisiones del proyecto.

Decisiones técnicas a fundamentar

- ¿La aplicación se despliega como una única task o múltiples tasks tras el ALB? Si es múltiple, ¿cómo se comporta Shlink frente al cache local de cada proceso?
- ¿Qué endpoint utiliza el ALB para el health check y qué umbrales de healthy/unhealthy se configuran?
- ¿Cómo se inicializa el schema de la base de datos en el primer despliegue y cómo se aplican migraciones futuras sin downtime?
- ¿Las API keys de Shlink se almacenan en SSM Parameter Store o en Secrets Manager? ¿Cómo se rotan?

Escenario B — Pasarela de webhooks

Aplicación recomendada: [Convoy](#) — gateway de webhooks con reintentos configurables, dead letter queue, rate limiting y métricas Prometheus integradas. Requiere PostgreSQL y Redis.

Componentes mínimos a desplegar

- Container de Convoy (servidor + worker — pueden correr en la misma task o separados).
- RDS PostgreSQL para configuración y registro de eventos.
- ElastiCache Redis (o Redis en container, si se justifica el trade-off) para la cola de eventos.

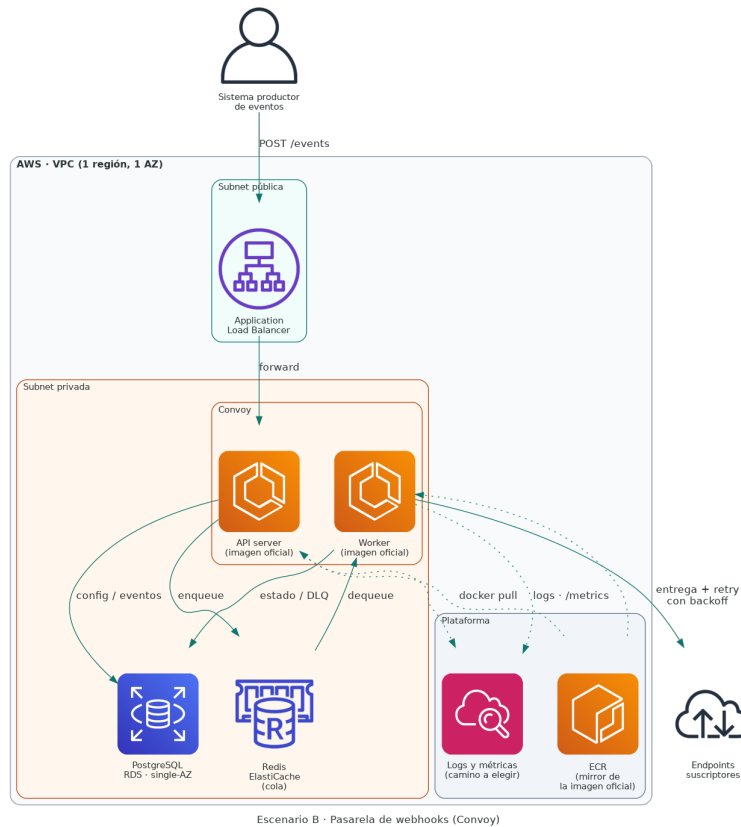
Métricas relevantes para SLOs

Profundidad de la cola, latencia p95 de entrega, tasa de éxito por endpoint, eventos enviados a la dead letter queue.

Documentación clave

- Configuración por archivo y variables de entorno:
<https://docs.getconvoy.io/configuration/configuration-file>
- Despliegue con Docker: <https://docs.getconvoy.io/deployment/manual/docker>
- Endpoint de métricas Prometheus: <https://docs.getconvoy.io/configuration/instrumentation>

Topología sugerida



Topología sugerida del escenario B. La separación de server y worker en tasks distintas, el dimensionamiento y la estrategia de retry son decisiones del proyecto.

Decisiones técnicas a fundamentar

- ¿Server y worker se despliegan en la misma task definition o por separado? ¿Qué implica para el escalado independiente de cada uno?
- Si el worker no puede entregar eventos durante un periodo prolongado, ¿qué tan grande puede crecer el backlog antes de saturar la cola? ¿Qué política de eviction o backpressure aplica?
- ¿La cola se implementa con Redis (ElastiCache) o con SQS? Si se elige SQS, ¿qué adaptaciones requiere la configuración de Convoy y qué se gana o se pierde?
- ¿Cómo se inspecciona y reprocesa la dead letter queue en operación normal? Esta decisión alimenta el runbook obligatorio.
- Entre latencia de entrega y tasa de éxito, ¿cuál es el SLI prioritario para este servicio? ¿Por qué?

Escenario C — Procesador de imágenes

Aplicación recomendada: [imgproxy](#) — servidor de procesamiento de imágenes en tiempo real, escrito en Go, con soporte nativo para fuentes en S3, métricas Prometheus integradas (variable `IMGPROXY_PROMETHEUS_BIND`) y health check en `/health`.

Componentes mínimos a desplegar

- Container de `darthsim/imgproxy` o `ghcr.io/imgproxy/imgproxy`.
- Bucket S3 de origen con las imágenes.
- (Opcional) Bucket S3 de cache para reducir el reprocesamiento.

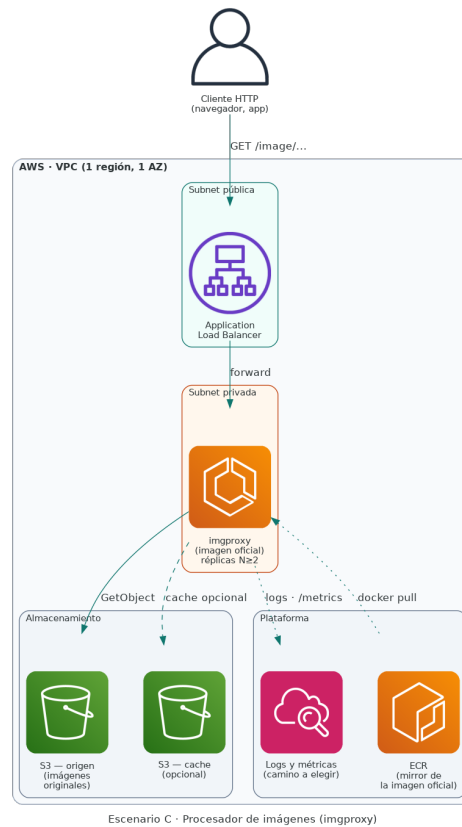
Métricas relevantes para SLOs

Latencia de procesamiento p95, tasa de errores, throughput, uso de memoria por worker.

Documentación clave

- Variables de entorno: <https://docs.imgproxy.net/configuration/options>
- Integración con S3: https://docs.imgproxy.net/serving_files_from_s3
- Métricas Prometheus: <https://docs.imgproxy.net/monitoring/prometheus>

Topología sugerida



Topología sugerida del escenario C. El número de réplicas, los límites de CPU/memoria, la política de cache y la organización de los buckets son decisiones del proyecto.

Decisiones técnicas a fundamentar

- imgproxy es CPU-bound por naturaleza. ¿Cuántas réplicas se ejecutan y bajo qué política de autoscaling? ¿Qué métrica dispara el escalado?
- ¿Las imágenes originales se exponen al público vía URL firmada de S3, vía proxy de imgproxy o de otra forma? ¿Cómo se previene el hotlinking?
- ¿Qué permisos exactos necesita el Task Role para acceder al bucket de origen? Listar las acciones IAM mínimas.
- ¿Existe un cache delante de imgproxy (CloudFront, otro bucket S3) o se procesa cada solicitud en caliente? ¿Cómo afecta esto al SLO de latencia y al costo?
- ¿Cómo se valida el comportamiento bajo carga real antes del go-live?

Escenario D — Programador de tareas

Aplicación recomendada: [Dkron](#) — sistema de scheduling distribuido escrito en Go, con REST API, almacenamiento en BoltDB o PostgreSQL, métricas Prometheus integradas y panel web para administración.

Componentes mínimos a desplegar

- Container de Dkron (modo server y, opcionalmente, una task adicional como agent para los runs).
- RDS PostgreSQL para persistencia de los jobs y su histórico.
- (Opcional) Storage S3 para los outputs.

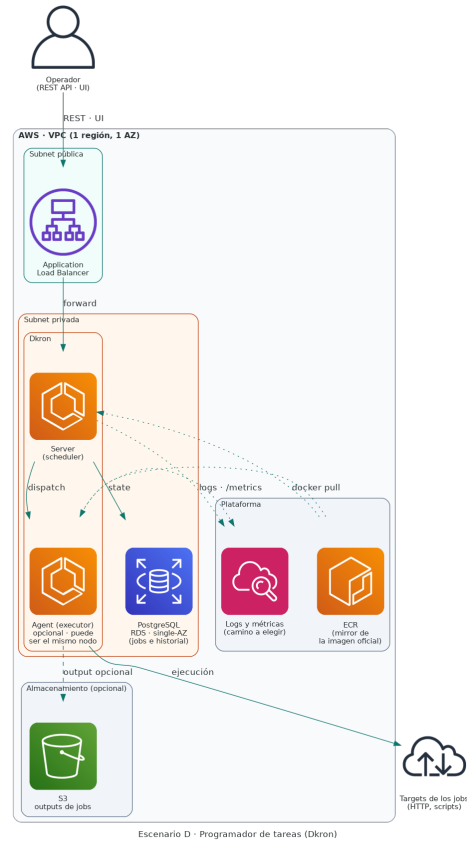
Métricas relevantes para SLOs

Jobs ejecutados a tiempo, drift de horario, jobs fallidos, duración p95 por tipo de job.

Documentación clave

- Despliegue con Docker: <https://dkron.io/docs/basics/getting-started/>
- REST API: <https://dkron.io/api/>
- Métricas: <https://dkron.io/docs/basics/configuration/#telemetry>

Topología sugerida



Topología sugerida del escenario D. La separación entre server y agent, el modo de almacenamiento (BoltDB local frente a PostgreSQL) y la persistencia de outputs son decisiones del proyecto.

Decisiones técnicas a fundamentar

- ¿Un único nodo Dkron o un cluster? Para el alcance del proyecto, ¿qué aporta operar en cluster?
- ¿El almacenamiento de jobs es BoltDB local (efímero al recrearse la task) o PostgreSQL (persistente)? ¿Qué se gana en cada caso?
- ¿Cómo se mide el drift entre la hora programada y la hora real de ejecución? ¿Qué métrica registra ese delta?
- ¿Cómo se previene la ejecución duplicada de un job si Dkron reinicia mientras el job estaba corriendo?
- ¿Qué política de timeout aplica a los jobs y qué pasa con un job que excede ese timeout?

Si se propone una aplicación distinta

Es válido proponer otra aplicación open source siempre que cumpla **todas** estas condiciones:

- Licencia permisiva (MIT, Apache 2.0, BSD, MPL 2.0 o similar).
- Imagen Docker oficial publicada por el proyecto.
- API HTTP documentada o protocolo de red estándar.
- Componente con estado persistente (base de datos, S3, cola).
- Métricas exportadas en formato Prometheus (nativas o vía exporter mantenido).
- No es un sitio web de presentación de dos capas.

La justificación va al reporte (sección 6.2 A).

3

Estrategia de integración del repositorio open source

El objetivo no es forkear ni modificar la aplicación. Es **operarla** desde la imagen oficial. Pasos sugeridos:

1. **Crear el repositorio del proyecto** en GitHub. Este repositorio contiene la infraestructura, el pipeline y la observabilidad. La aplicación se referencia por su imagen Docker, no por su código.
2. **Estudiar la documentación de despliegue** del proyecto open source elegido: variables de entorno requeridas y opcionales, dependencias externas (base de datos, cache, almacenamiento), endpoint de health check, endpoint de métricas (puerto, path, formato), esquema de migraciones de base de datos cuando aplica.
3. **Construir un docker-compose.yml local** que levante la aplicación junto con sus dependencias. Sirve para validar la configuración antes de migrar a AWS y para desarrollar contra una réplica fiel del entorno productivo. Usar la imagen oficial pinneada a una versión específica (jamás :latest) y declarar las variables relevantes en un .env.example versionado.
4. **Definir las variables de configuración para producción** en Terraform. Variables no sensibles (puertos, flags, niveles de log) como environment en la task definition. Variables sensibles (URLs de conexión con credenciales, API keys) en SSM Parameter Store o Secrets Manager, inyectadas vía secrets.
5. **Desplegar la imagen oficial** en la opción de cómputo elegida (ECS Fargate, EKS o EC2 + Docker). El campo image apunta a la imagen pública o a una copia mirror en ECR si se prefiere control sobre la disponibilidad.
6. **Configurar el scrape de métricas** desde el sistema de observabilidad hacia el endpoint /metrics (o el path correspondiente). Esta es una de las tareas centrales del proyecto: la aplicación expone métricas, pero la recolección, los dashboards y las alertas se diseñan como parte del trabajo.
7. **Pinnear todas las versiones:** imagen Docker, versión de PostgreSQL en RDS, versión de Redis en ElastiCache. Documentarlas en el README y en el reporte.

Aviso Replicación recomendada a ECR

La práctica recomendada es replicar la imagen oficial a un repositorio ECR propio (mediante `docker pull + docker tag + docker push`) para que el entorno productivo no dependa de la disponibilidad de Docker Hub o GHCR. Esta replicación se documenta en el reporte como una decisión consciente.

Pista No modificar el código upstream

No se permite hacer fork del repositorio open source ni modificar su código. El alcance del proyecto cubre la infraestructura, el pipeline, la observabilidad y la operación. Si encuentran un bug, repórtelo upstream y documenten el workaround.

4

Alcance

El alcance del proyecto está acotado a los siguientes límites:

- Una sola región y una sola Availability Zone son suficientes.
- Un solo entorno (prod o staging).
- Una sola cuenta de AWS.
- Sin custom domains, sin certificados ACM, sin Route 53. El DNS por defecto del ALB es suficiente.
- Sin RDS multi-AZ, sin Helm chart propio, sin canary, sin blue-green, sin IRSA, sin Secrets Manager con CSI driver.

Cualquier capacidad fuera de este alcance solo agrega valor si está justificada en el reporte y si no compromete la entrega del resto.

5 Requisitos del proyecto

5.1 Infraestructura como código (Terraform)

- Código nuevo, organizado en módulos propios. Los talleres del bootcamp sirven como referencia de estructura, no como código de partida.
- Mínimo dos módulos propios (por ejemplo `network/` y `compute/`) o un módulo propio combinado con un módulo del registro público (`terraform-aws-modules/vpc/aws` cuenta).
- State remoto en S3 con locking (DynamoDB o `use_lockfile` nativo).
- IAM con mínimo privilegio efectivo: cada Task Role o Role tiene únicamente los permisos necesarios.
- Tags obligatorios en todos los recursos: `Project`, `Environment`, `Owner`, `ManagedBy=Terraform`.
- `terraform destroy` debe ejecutarse limpio, sin recursos huérfanos.

5.2 Cómputo y red

Tres opciones válidas para el cómputo. Elegir una y justificarla:

- **Opción A — ECS Fargate:** orquestación sin servidores, integración nativa con ALB, task definitions en Terraform.
- **Opción B — EC2 con Docker Compose desplegado por Ansible:** aplica el patrón EC2 + Ansible visto en los talleres a una infraestructura nueva.
- **Opción C — EKS:** orquestación Kubernetes gestionada. Mayor complejidad operativa y costo.

Independiente de la opción:

- ALB público delante del componente que reciba tráfico HTTP.
- Componentes de aplicación en subnet privada.
- Persistencia: RDS PostgreSQL en subnet privada, single-AZ. Cache o cola según el escenario, vía ElastiCache Redis o SQS.
- Security groups restrictivos: tráfico abierto a Internet solo en el ALB y, si aplica, en el SSH del bastion. La base de datos solo acepta tráfico desde el security group de la aplicación.

5.3 CI/CD (GitHub Actions)

Pipeline propio en `.github/workflows/`. Los talleres son referencia de estructura, no plantilla a copiar. Mínimos no negociables:

1. **Validación de IaC:** `terraform fmt -check`, `terraform validate`, `tflint`, **Checkov** o **tfsec**. Si el escaneo de seguridad bloquea, se corrige el hallazgo o se agrega a una skip list documentada con la razón.
2. **Replicación de la imagen oficial a ECR:** pull de la imagen open source pinneada y push a un repositorio ECR propio. Se hace en el pipeline o en una tarea programada.

3. **Escaneo de la imagen replicada con Trivy:** el job falla si hay vulnerabilidades HIGH o CRITICAL sin excepción documentada en `.trivyignore`.
4. **Plan de Terraform en pull requests** (sin apply).
5. **Apply de Terraform y deploy del servicio en push a main**, en ese orden, gateado por el environment `production` con aprobación manual configurable.
6. **Workflow separado de destroy** con confirmación tipeada, equivalente al patrón `destruir.yaml` visto en los talleres.
7. **Concurrency configurada** para evitar dos applies simultáneos sobre el mismo state.
8. Ningún paso crítico con `continue-on-error: true`. Si el pipeline necesita tolerar un error, la decisión se documenta.

La estructura del pipeline (workflows separados o uno solo, coordinación con `workflow_run` o `needs`) es una decisión del proyecto y se justifica en el reporte.

5.4 Observabilidad

Se elige una de tres aproximaciones y se justifica:

- **Camino 1 — Self-hosted Prometheus + Grafana + Loki:** instalación nueva, dimensionada al servicio.
- **Camino 2 — CloudWatch nativo:** Container Insights, CloudWatch Metrics, Alarms, Logs Insights y Dashboards.
- **Camino 3 — Servicios gestionados:** Amazon Managed Prometheus (AMP) + Amazon Managed Grafana (AMG).

Mínimos independientes del camino:

- Scrape o ingestión efectiva de las métricas custom expuestas por la aplicación open source.
- Mínimo dos SLOs definidos sobre métricas de aplicación o de negocio relevantes al escenario. Los SLOs no se basan únicamente en CPU/RAM.
- Mínimo un dashboard que aplique el método **RED** (Rate, Errors, Duration) o **USE** según corresponda al servicio.
- Mínimo dos alertas configuradas que disparen ante violaciones de los SLOs. Las alertas se prueban antes de la entrega y la evidencia queda en el reporte.
- Logs centralizados y buscables. Es posible buscar un identificador (por ejemplo `request_id`, `job_id` o equivalente) y correlacionarlo con un evento en las métricas del mismo timeframe.

5.5 Seguridad

- Sin secretos en el repositorio. Las credenciales y URLs de conexión sensibles se almacenan en SSM Parameter Store o Secrets Manager y se inyectan en runtime.
- Trivy en el pipeline sobre la imagen replicada a ECR.
- Checkov o tfsec en el pipeline sobre el código Terraform.
- Security groups restrictivos según la sección 5.2.

6

Entregables

6.1 Repositorio en GitHub

Estructura sugerida:

Estructura del repositorio

```
.
|-- compose/ # docker-compose.yml local + .env.example
|-- infra/
| |-- modules/
| | |-- network/
| | `-- compute/
| |-- main.tf
| `-- backend.tf # state remoto en S3
|-- observability/ # configuracion del camino elegido (5.4)
|-- .github/workflows/
| |-- ci-cd.yaml # o varios archivos coordinados
| `-- destruir.yaml
|-- docs/
| |-- arquitectura.md # diagrama y descripcion
| `-- runbook.md # un runbook (seccion 6.3)
|-- REPORTE.md # reporte tecnico (seccion 6.2)
`-- README.md
```

El README.md contiene:

- Propósito del proyecto y escenario elegido, con enlace al repositorio open source de la aplicación y la versión pinneada.
- Diagrama de arquitectura.
- Instrucciones para levantar el entorno local (docker compose up).
- Instrucciones para desplegar desde cero (orden de comandos, variables requeridas).
- Sección de evidencias del funcionamiento, según se detalla en la sección 6.4.

6.2 Reporte técnico (REPORTE.md)

Aviso Reporte sin asistencia de IA generativa

El reporte es el entregable de mayor peso. Está escrito por cada estudiante con sus propias palabras, sin asistencia de IA generativa y sin copiar texto de la documentación oficial. El propósito del reporte es evidenciar la comprensión personal del estudiante sobre cómo se resuelve el problema implementado: lo que se evalúa es la calidad del razonamiento, no la prosa. Extensión: mínimo 2.000 palabras, máximo 5.000.

Estructura obligatoria:

A — El problema y la arquitectura (≈ 1 página)

- Escenario elegido y justificación de la aplicación open source seleccionada.
- Diagrama de arquitectura comentado.
- Por cada bloque (cómputo, base de datos, cache o cola, observabilidad, CI/CD): qué se eligió, por qué, qué alternativas se consideraron y bajo qué condiciones la decisión se revisaría.

B — Conceptos clave (≈ 2-3 páginas)

Cada concepto se desarrolla entre media página y una página, con palabras propias del estudiante. Lo evaluado en esta sección es la calidad de la explicación, no el código.

1. Diferencia entre Infraestructura como Código y gestión de configuración. Aplicación concreta de Terraform y Ansible en este proyecto.
2. Por qué la containerización mejora el ciclo de CI/CD frente al modelo "EC2 + Ansible" visto en los talleres.
3. Diferencia entre Continuous Integration, Continuous Delivery y Continuous Deployment. Nivel implementado en el proyecto y razón.
4. SLI, SLO y error budget. Justificación de los valores numéricos elegidos.
5. Mínimo privilegio en IAM. Aplicación concreta, dificultades encontradas y permisos que fue necesario ampliar con su justificación.
6. State remoto y locking en Terraform. Consecuencia de un apply concurrente sin lock.
7. Trade-offs del componente asíncrono o de estado introducido por el escenario elegido (cola, cache, scheduler, almacenamiento de objetos): qué problema resuelve y qué problemas operativos nuevos introduce.

C — Problemas encontrados y resolución (≈ 1 página)

Entre tres y cinco problemas reales enfrentados durante el desarrollo. Por cada uno:

- Síntoma observado.
- Método de investigación: logs, métricas, comandos utilizados.
- Causa raíz.
- Solución aplicada.
- Acciones preventivas para el futuro.

D — Costos (≈ media página)

- Tabla con los recursos desplegados y el costo mensual estimado (AWS Pricing Calculator o Infracost).
- Dos optimizaciones concretas que se aplicarían si el servicio escalara a producción real.

E — Reflexión final (≈ media página)

- Funcionalidades que se dejarían para una segunda iteración.
- Componente del proyecto que mejor demuestra competencias de un puesto DevOps Mid.
- Componente que sería sustituido en un entorno productivo real, indicando la alternativa.

6.3 Runbook (docs/runbook.md)

Un runbook ejecutable por una persona ajena al proyecto, dedicado a una operación realista del escenario elegido. Ejemplos:

- Rollback de un deploy fallido.
- Restauración de la base de datos desde un snapshot.
- Drenaje y reproceso de la dead letter queue (escenario B).
- Invalidación del cache después de un cambio de schema (escenario A).
- Recuperación tras la pérdida del nodo primario (escenarios A, B, D).

6.4 Evidencias del funcionamiento

El proyecto se evalúa principalmente sobre el material escrito entregado en el repositorio. Para que el revisor pueda verificar que la solución funciona sin necesidad de desplegarla, el repositorio incluye, en una sección dedicada del README.md o en docs/evidencias.md, las siguientes capturas:

1. Listado de ejecuciones recientes del workflow en la pestaña Actions de GitHub, mostrando un pull request validado, un merge a main y un deploy automatizado posterior.
2. La aplicación atendiendo un caso de uso del escenario (respuesta de un endpoint clave o pantalla de la UI, según corresponda).
3. El dashboard de observabilidad con los SLOs definidos y su estado saludable.
4. La alerta disparada de forma intencional, junto con una descripción breve de cómo se generó la violación del SLO.

Las capturas tienen marca de tiempo legible y muestran el contexto suficiente para reconocer el recurso (URL, nombre del workflow, identificador de la task, nombre del dashboard).

6.5 Video explicativo (opcional)

Como complemento **opcional** al material principal, se sugiere grabar un video de al menos 10 minutos donde el estudiante recorra el proyecto y fundamente sus decisiones. La ausencia de este entregable no afecta la calificación; presentarlo refuerza la evidencia de comprensión personal y puede inclinar la evaluación a favor del estudiante en casos limítrofes, especialmente sobre la sección de conceptos clave del reporte. Se publica como enlace privado (YouTube no listado, Vimeo, Loom) referenciado desde el README.md. No requiere edición ni producción profesional: el objetivo es la explicación.

Estructura sugerida:

- Recorrido por la arquitectura desplegada, justificando la elección de cada componente.
- Demostración del pipeline ejecutándose end-to-end: pull request, validaciones, merge a main, deploy.
- Muestra del dashboard de observabilidad y de los SLOs definidos.
- Generación intencional de una violación del SLO y entrega de la alerta correspondiente.
- Reflexión sobre los desafíos encontrados durante la operación y cómo se resolvieron.

7 Reglas de entrega

- **El proyecto es individual.** Cada estudiante entrega su propio repositorio y su propio reporte. No hay defensa oral ni presentación en vivo: la evaluación se realiza sobre el material entregado. Opcionalmente puede acompañarse de un video explicativo grabado, según la sección 6.5.
- **Se permite y se fomenta el intercambio de ideas entre estudiantes:** discutir trade-offs, comparar enfoques, comentar arquitecturas, resolver dudas conceptuales. No se permite compartir código del repositorio ni reproducir total o parcialmente el reporte de otra persona.
- **El reporte se escribe sin asistencia de IA generativa.** El propósito del reporte es demostrar la comprensión personal del estudiante sobre cómo se resuelve el problema implementado; esa comprensión es lo que se evalúa. El uso de IA durante el desarrollo del código del proyecto es aceptable y refleja una práctica habitual en la industria. Aplicado a la redacción del reporte invalida ese propósito; si se detecta, la sección se califica en cero.
- El código de los talleres del bootcamp es referencia de estructura, no código de partida. Importar módulos o pipelines del taller directamente no cumple el requisito de infraestructura nueva.
- AWS Free Tier cubre la mayoría del consumo. NAT Gateway, RDS y ElastiCache son los componentes con costo continuo: se apagan cuando no están en uso, mediante el workflow de destroy del proyecto.
- Al finalizar, ejecutar `terraform destroy` (o el workflow correspondiente) para evitar costos residuales.

8

Criterios de evaluación

Bloque	Peso
Reporte técnico (claridad de decisiones, dominio conceptual, problemas reales)	45%
CI/CD (pipeline reproducible, escaneos, deploy automático)	25%
Containerización y despliegue en AWS (cómputo, ALB, IAM, IaC)	20%
Observabilidad (métricas custom, dashboard, SLOs y alertas verificadas)	10%

El reporte concentra el mayor peso porque la diferencia entre un perfil junior y un perfil mid-level se manifiesta en la capacidad de fundamentar técnicamente las decisiones tomadas. Sin defensa oral, el reporte es la única vía para evidenciar esa comprensión, y por eso se evalúa con criterio estricto sobre la solidez del razonamiento.

9

Mapeo a módulos del bootcamp

Módulo	Cómo aparece en el proyecto
1 — Fundamentos	Git, branching, pull requests, cultura DevOps
2 — AWS	VPC, ALB, ECS/EKS o EC2, RDS, ECR, IAM, security groups, SQS o ElastiCache
3 — Contenedores	Imagen oficial pinneada, docker-compose local, orquestación
4 — CI/CD	GitHub Actions: validación, escaneo, build, deploy con aprobación manual
5 — IaC	Terraform con módulos propios, state remoto, providers actualizados
6 — DevSecOps	Trivy, Checkov o tfsec, secretos en SSM o Secrets Manager
7 — FinOps	Tagging consistente, sección de costos en el reporte
8 — SRE	SLOs específicos del escenario, runbook, alertas verificadas
9 y 10	Fuera de alcance

10 Preguntas frecuentes

¿Es válido elegir GCP o Azure?

No. AWS es la nube objetivo del proyecto.

¿Se puede proponer una aplicación open source distinta a las cuatro recomendadas?

Sí, siempre que cumpla las condiciones listadas al final de la sección 2.

¿Es obligatorio replicar la imagen oficial a ECR?

Es la práctica recomendada y la más cercana a un entorno productivo. Si se decide consumir la imagen directamente desde Docker Hub o GHCR, debe documentarse el riesgo asociado a esa dependencia externa.

¿Es necesario reescribir el código de la aplicación?

No. El alcance es la operación del servicio, no su desarrollo. Si se encuentran bugs upstream, se reportan al proyecto original y se documentan los workarounds en el repositorio del proyecto.

¿Es necesario montar Prometheus y Grafana?

Solo si se elige el camino 1 de la sección 5.4. Los caminos 2 (CloudWatch) y 3 (AMP/AMG) son válidos y, según el escenario, más adecuados.

¿Es obligatorio usar Ansible?

Solo si se elige la opción B de la sección 5.2 (EC2 con Docker Compose). En ECS Fargate y EKS no aplica.

¿GitHub Actions o Jenkins?

GitHub Actions, por integración nativa con el repositorio y por estandarización con el resto del bootcamp.

¿Es obligatorio un Helm chart propio?

No. Para EKS, manifiestos planos o un chart mínimo cumplen el requisito.

¿Qué pesa más, el código o el reporte?

El reporte. Sin reporte, el código no se evalúa. Sin código, el reporte tampoco. En el desempate, se evalúa la calidad de la fundamentación.

¿Puedo usar IA para redactar el reporte?

No. La regla está detallada en la sección 7. La IA es bienvenida durante el desarrollo del código del proyecto; el reporte debe escribirse por el estudiante, en sus propias palabras, porque su único propósito es evidenciar comprensión personal.

¿Puedo usar IA para corregir ortografía o pulir la redacción del reporte?

Para corrección de ortografía y gramática es aceptable. Para parafrasear, expandir argumentos, generar estructura o sustituir contenido, no. La distinción es: la IA puede revisar lo que ya está escrito por el estudiante, pero no puede generarlo.

¿Cómo se verifica que el reporte fue escrito sin IA?

No hay un test infalible, pero hay señales claras (tono uniformemente impersonal, ausencia de errores menores típicos del español de cada autor, ejemplos genéricos en lugar de problemas reales del proyecto, falta de referencias a las decisiones específicas del repositorio entregado). Cuando esas señales se acumulan, la sección se califica en cero.